

今月のフロントエンド

フロントエンド エンジニア集会 2026 年 7 月

"今月のフロントエンド" とは

今月あったフロントエンドのニュースを, 以下のフォーマットで紹介します.

- ニュースがあった技術について, その名前かキーワード
- その技術に関する解説 (3 行目安)
- 何があったかを解説

取り上げないもの

- 特定フレームワークに関するバージョンアップ (例外あり)
- AI 単品のニュース

ECMAScript 2026

2026/06/30



"ECMAScript" とは

- JavaScript のコア仕様を定める標準規格 (ECMA-262)
- TC39 が策定し, 毎年 6 月頃の Ecma 総会で新版が承認される
- 各ブラウザや Node.js は, この仕様に沿って JavaScript を実装する

ECMAScript 2026 のニュース

ES2026 (第 17 版) が正式承認

- 6/30 の Ecma 総会 (第 131 回) で **ECMA-262 第 17 版** として承認
- `Math.sumPrecise()` / `Map.getOrInsert()` / `Error.isError()` など
「毎回自前で書いていた処理」を置き換える実用系メソッドが多数
- `Uint8Array` ⇔ base64 / hex の相互変換が標準化
- `Array.fromAsync()` / `Iterator.concat()` / `JSON.rawJSON()` も標準入り

ただし Temporal は入らなかった

- 3 月に Stage 4 入りした **Temporal** (次世代の日時 API) は、仕様編集の締切に間に合わず 今回の版には含まれない
- 正式な仕様入りは **ES2027** に持ち越し
- Stage 4 なので方向性は確定済み、各環境の実装は先行して進んでいる
- ※ 5 月号では「ES2026 に正式採用」と紹介していました。訂正します 📄👤

Google Chrome

2026/06/30



"Google Chrome" とは

- Google が開発する, デスクトップで圧倒的シェアを持つ Web ブラウザ
- レンダリングエンジンは Blink, 新しい CSS / Web API の実装を積極的に主導
- Edge など他の Chromium 系ブラウザの基盤にもなっている

Google Chrome のニュース

Chrome 150 — "文字を枠にフィットさせる" text-fit が登場

- `text-fit` : 見出しなどの フォントサイズを枠の幅に自動で合わせる
- `background-clip: border-area` : CSS だけでグラデーション枠線が引ける
- `focusgroup` 属性 : 矢印キーによるフォーカス移動を 宣言的に 書ける
- ほかに `flex-wrap: balance`, `polygon()` の角丸対応など CSS 強化が多数

何が嬉しいのか

- 「枠いっぱいの見出し」は従来 **JS** ライブラリや手計算の **clamp()** が必要だった
 - CSS 一発で, リサイズにも自動で追従する
- グラデーション枠線も従来は **border-image** や疑似要素のハックが定番だった
- **focusgroup** はこれまで **JS** での **roving tabindex** 実装 が必要だった領域
 - ツールバーやタブ UI のキーボード操作がマークアップだけで書ける

他ブラウザとの互換性

機能	Chrome / Edge	Safari	Firefox
<code>text-fit</code>	 (150～)		
<code>background-clip: border-area</code>	 (150～)	 (18.2～)	
<code>focusgroup</code>	 (150～)		

- `text-fit` / `focusgroup` は **Chromium 単独先行** (標準化は議論中)
- 文字サイズや枠線は, 非対応ブラウザでは **通常表示に落ちるだけ**
→ 段階的強化で使える

TypeScript

2026/07/08



"TypeScript" とは

- Microsoft が開発する, JavaScript に静的型を加えた言語
- 現代のフロントエンド開発では事実上の標準になっている
- コンパイラは長らく **TypeScript 自身で実装され**, Node.js 上で動いてきた

TypeScript のニュース

TypeScript 7.0 が正式リリース — コンパイラを Go で書き直し, 約 10 倍高速に

- コンパイラと言語サービスを **Go** へ**完全移植** (Project Corsa)
- フルビルドは **典型的に 8~12 倍** 高速化
 - VS Code コードベースの型チェック: **125.7 秒 → 10.6 秒**
- アルゴリズムとデータ構造をそのまま移植し, **型チェックの結果は変えない** 方針

移行にあたって

- 2025/03 の「ネイティブ化で 10 倍速くする」予告 (Project Corsa) がついに完成形に
- ネイティブ化の恩恵は エディタの補完・エラー表示の体感速度 にも及ぶ
- JS 実装の 6.x 系も当面は並走 するが、今後の主軸は 7 系

ただし、波及はまだこれから

TS 7.0 は "プログラムから使う API" を持たずに出荷された

- 旧実装では周辺ツールが TS の **Compiler API** を通じて型情報を得ていた
- Go 実装の安定 API 提供は **TS 7.1 (2026 年 10 月頃見込み)** から
- つまり **Compiler API** に依存するツールは軒並み、まだ **TS 7** に乗れない
- `vue-tsc` (Volar) は `.vue` の型解決が API 依存 → 未対応,
Vue / Svelte / Astro / MDX のワークフローは **当面 TS 6 系**に留まることに

typescript-eslint も同様

- 型情報を使う lint ルール も Compiler API 経由 → TS 7 では動かない
- typescript-eslint の TS 7 対応 issue は「7.1 の API 待ち」として一旦クローズ
- 当面の公式推奨は「ビルドは TS 7, ESLint 用に TS 6 を並走」させる構成
- 一方, Rust 製 Oxc が tsgolint を fork し 型付き lint を 20~40 倍高速化する
「API を介さない」代替ルートも進行中

npm

2026/07/08

npm

"npm" とは

- Node.js 標準のパッケージマネージャ / 世界最大の JS パッケージレジストリ
- `npm install` 一発で依存を解決する, フロントエンド開発の生命線
- 現在は GitHub (Microsoft) が運営している

6 月に予告されていたこと

- 運営元の GitHub が 6/9, **v12** での破壊的変更 を予告していた
- 背景は, 近年多発する **postinstall** 経由のサプライチェーン攻撃
- `npm install` は依存パッケージの任意コードを 自動実行する 仕様で,
「入れた瞬間に感染する」攻撃の温床になっていた

npm のニュース

npm v12 が正式リリース — install スクリプトは既定で動かない時代へ

- 予告通り 7/8 にリリース.
`preinstall` / `install` / `postinstall` は 既定で実行されない
- 実行を許すパッケージだけ `npm approve-scripts` で **allowlist** に明示許可
- **git 依存 / リモート URL 依存**も既定では解決されないように
- CLI 16 年の歴史で初めて, 「入れる」と「実行する」が分離された

移行の実務

- `npm approve-scripts --allow-scripts-pending` でスクリプト実行を求めている依存を列挙 できる
- 信頼するものだけ許可し, allowlist を `package.json` にコミットする運用
- npm **11.16** 以降では警告付きで先行確認 できる (v12 に上げる前に棚卸しを)
- ネイティブビルドに依存するパッケージを使う CI は要チェック

今月のインシデント

フロントエンド エンジニア集会 2026 年 7 月

"今月のインシデント" とは

今月あったフロントエンド周辺のインシデントを、以下のフォーマットで紹介します。

- インシデントの起きた技術について、その名前かキーワード
- その技術に関する解説 (3 行目安)
- 何が起きたかを解説

取り上げないもの

- フロントエンドと無関係なエコシステム単独の事案 (PyPI のみ, Go のみ など)
- 個別 CVE レベルの脆弱性の発見・修正
- SaaS / クラウド事業者単体への攻撃 (npm を経由しないもの)

AsyncAPI

2026/07/14

"AsyncAPI" とは

- イベント駆動 API (Kafka / MQTT / WebSocket など) の仕様記述フォーマット
- 「OpenAPI の非同期版」にあたる OSS プロジェクトで, コード生成ツールも提供
- npm の `@asyncapi` 配下のパッケージは 合計 週 300 万 DL を超える

AsyncAPI のインシデント

@asynccapi の 4 パッケージが侵害, "import した瞬間" に動くマルウェア

- 7/14, リリース用 bot の資格情報が奪われ, 約 90 分間に 悪意ある 5 バージョン が公開
- ペイロードは **install** 時ではなく **import / require** された瞬間に実行
- 認証情報を窃取し, 第 2 段のマルウェア **Miasma** を IPFS (分散型ファイル共有ネットワーク) から取得する多段構成

侵害の経路

- 攻撃者は generator リポジトリに **大量の PR** を目くらましとして投げた
- その中に GitHub Actions の設定不備を突く PR を紛れ込ませる ("pwn request")
- CI がその PR を実行してしまい, リリース **bot** のトークンが流出
- 奪ったトークンで, 正規の手順どおりに悪意あるバージョンが公開された

なぜ象徴的なのか

- npm v12 が postinstall を塞いだ **わずか 6 日後** のインシデント
- 実行タイミングを **install 時** → **import 時** に移され,
`approve-scripts` の allowlist では **防げない** 手口だった
- 「インストールさえ安全なら OK」ではない —
依存コードそのものの信頼 が問われるフェーズに入った

我々はどう守るか

- 依存の更新は即時に取り込まず, 数日置いてから 上げる (cooldown 運用)
- lockfile を固定し, CI では `npm ci` を使う
- 自分たちの CI も攻撃対象: `pull_request_target` など **Actions** の 権限設定を点検
- 「npm v12 で安心」ではなく, 多層で守る前提を持つ

おわり